14
Computational Geometry

Geometric computing has been become increasingly important in applications such as computer graphics, robotics, and computer-aided design, because shape is an inherent property of real objects. But most real-world objects are not made of lines which go to infinity. Instead, most computer programs represent geometry as arrangements of line segments. Arbitrary closed curves or shapes can be represented by ordered collections of line segments or *polygons*.

Computational geometry can be defined (for our purposes) as the geometry of discrete line segments and polygons. It is a fun and interesting subject, but one not typically taught in required college courses. This gives the ambitious student who learns a little computational geometry a leg up on the competition, and a window into a fascinating area of algorithms still under active research today. Excellent books on computational geometry are available [O'R00, dBvKOS00], but this chapter should be enough to get you started.

14.1 Line Segments and Intersection

A *line segment* s is the portion of a line l which lies between two given points inclusive. Thus line segments are most naturally represented by pairs of endpoints:

```
typedef struct {
        point p1,p2;            /* endpoints of line segment */
} segment;
```

The most important geometric primitive on segments, testing whether a given pair of them intersect, proves surprisingly complicated because of tricky special cases that

arise. Two segments may lie on parallel lines, meaning they do not intersect at all. One segment may intersect at another's endpoint, or the two segments may lie on top of each other so they intersect in a segment instead of a single point.

This problem of geometric special cases, or *degeneracy*, seriously complicates the problem of building robust implementations of computational geometry algorithms. Degeneracy can be a real pain in the neck to deal with. Read any problem specification carefully to see if it promises no parallel lines or overlapping segments. Without such guarantees, however, you had better program defensively and deal with them.

The right way to deal with degeneracy is to base all computation on a small number of carefully crafted geometric primitives. In Chapter 13, we implemented a general line data type that successfully dealt with vertical lines; those of infinite slope. We can reap the benefits by generalizing our line intersection routines to line segments:

```
bool segments_intersect(segment s1, segment s2)
{
      line l1,l2;             /* lines containing the input segments */
      point p;                /* intersection point */

      points_to_line(s1.p1,s1.p2,&l1);
      points_to_line(s2.p1,s2.p2,&l2);

      if (same_lineQ(l1,l2))  /* overlapping or disjoint segments */
              return( point_in_box(s1.p1,s2.p1,s2.p2) ||
                      point_in_box(s1.p2,s2.p1,s2.p2) ||
                      point_in_box(s2.p1,s1.p1,s1.p2) ||
                      point_in_box(s2.p1,s1.p1,s1.p2) );

      if (parallelQ(l1,l2)) return(FALSE);

      intersection_point(l1,l2,p);

      return(point_in_box(p,s1.p1,s1.p2) && point_in_box(p,s2.p1,s2.p2));
}
```

We will use our line intersection routines to find an intersection point if one exists. If so, the remaining question is whether this point lies within the region defined by our line segments. This is most easily tested by establishing whether the intersection point lies in the bounding box around each line segment, which is defined by the endpoints of each segment:

```
bool point_in_box(point p, point b1, point b2)
{
      return( (p[X] >= min(b1[X],b2[X])) && (p[X] <= max(b1[X],b2[X]))
           && (p[Y] >= min(b1[Y],b2[Y])) && (p[Y] <= max(b1[Y],b2[Y])) );
}
```

Segment intersection can also be cleanly tested using a primitive to check whether three ordered points turn in a counterclockwise direction. Such a primitive is described in the next section. However, we find the `point_in_box` method more intuitive.

14.2 Polygons and Angle Computations

Polygons are closed chains of non-intersecting line segments. That they are closed means the first vertex of the chain is the same as the last. That they are non-intersecting means that pairs of segments meet only at endpoints.

Polygons are the basic structure to describe shapes in the plane. Instead of explicitly listing the segments (or edges) of polygon, we can implicitly represent them by listing the n vertices in order around the boundary of the polygon. Thus a segment exists between the ith and $(i+1)$st points in the chain for $0 \leq i \leq n-1$. These indices are taken mod n to ensure there is an edge between the first and last point:

```
typedef struct {
        int n;                  /* number of points in polygon */
        point p[MAXPOLY];       /* array of points in polygon */
} polygon;
```

A polygon P is *convex* if any line segment defined by two points within P lies entirely within P; i.e., there are no notches or bumps such that the segment can exit and re-enter P. This implies that all internal angles in a convex polygon must be *acute*; i.e., at most 180° or π radians.

Actually computing the angle defined between three ordered points is a tricky problem. We can avoid the need to know actual angles in most geometric algorithms by using the *counterclockwise predicate* `ccw(a,b,c)`. This routine tests whether point c lies to the right of the directed line which goes from point a to point b. If so, the angle formed by sweeping from a to c in a counterclockwise manner around b is acute, hence the name of the predicate. If not, the point either lies to the left of $\overrightarrow{ab}$ or the three points are collinear.

These predicates can be computed using the `signed_triangle_area()` formula introduced in Section 13.2.3. Negative area results if point c is to the left of $\overrightarrow{ab}$. Zero area results if all three points are collinear. For robustness in the face of floating point errors, we compare it to a tiny constant ϵ instead of zero. This is an imperfect solution; building *provably* robust geometric code with floating point arithmetic is somewhere between difficult and impossible. However, it is better than nothing.

```
bool ccw(point a, point b, point c)
{
      double signed_triangle_area();

      return (signed_triangle_area(a,b,c) > EPSILON);
}
```

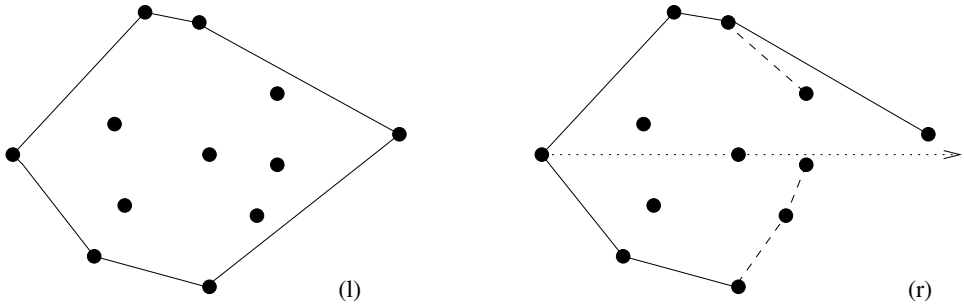


Figure 14.1. The convex hull of a set of points (l), with the change in hull due to inserting the rightmost point (r).

```
bool cw(point a, point b, point c)
{
      double signed_triangle_area();

      return (signed_triangle_area(a,b,c) < EPSILON);
}

bool collinear(point a, point b, point c)
{
      double signed_triangle_area();

      return (fabs(signed_triangle_area(a,b,c)) <= EPSILON);
}
```

14.3 Convex Hulls

Convex hull is to computational geometry what sorting is to other algorithmic problems, a first step to apply to unstructured data so we can do more interesting things with it. The *convex hull* $C(S)$ of a set of points S is the smallest convex polygon containing S, as shown in Figure 14.1(l).

There are almost as many different algorithms for convex hull as there are for sorting. The Graham's scan algorithm for convex hull which we will implement first sorts the points in either angular or left-right order, and then incrementally inserts the points into the hull in this sorted order. Previous hull points rendered obsolete by the last insertion are then deleted.

Our implementation is based on the Gries and Stojmenović [GS87] version of Graham scan, which sorts the vertices by *angle* around the leftmost-lowest point. Observe that

both the leftmost and lowest points *must* lie on the hull, because they cannot lie within some other triangle of points. We use the second criteria to break ties for the first, since there might be many different but equally leftmost points. Such considerations are necessary to achieve robustness with degenerate input.

The main loop of the algorithm inserts the points in increasing angular order around this initial point. Because of this ordering, the newly inserted point must sit on the hull of the thus-far-inserted points. This new insertion may form a triangle containing former hull points which now must be deleted. These points-to-be-deleted will sit at the end of the chain as the most recent surviving insertions. The deletion criteria is whether the new insertion makes an obtuse angle with the last two points on the chain – recall that only acute angles appear in convex polygons. If the angle is too large, the last point on the chain has to go. We repeat until a small enough angle is created or we run out of points. We can use our `ccw()` predicate to test whether the angle is too big:

```
point first_point;              /* first hull point */

convex_hull(point in[], int n, polygon *hull)
{
        int i;                  /* input counter */
        int top;                /* current hull size */
        bool smaller_angle();

        if (n <= 3) {           /* all points on hull! */
                for (i=0; i<n; i++)
                        copy_point(in[i],hull->p[i]);
                hull->n = n;
                return;
        }

        sort_and_remove_duplicates(in,&n);
        copy_point(in[0],&first_point);

        qsort(&in[1], n-1, sizeof(point), smaller_angle);

        copy_point(first_point,hull->p[0]);
        copy_point(in[1],hull->p[1]);

        copy_point(first_point,in[n]);  /* sentinel for wrap-around */
        top = 1;
        i = 2;

        while (i <= n) {
                if (!ccw(hull->p[top-1], hull->p[top], in[i]))
                        top = top-1;    /* top not on hull */
                else {
```

```
                        top = top+1;
                        copy_point(in[i],hull->p[top]);
                        i = i+1;
                }
        }

        hull->n = top;
}
```

The beauty of this implementation is how naturally it avoids *most* of the problems of degeneracy. A particularly insidious problem is when three or more input points are collinear, particularly when one of these points is the leftmost-lowest hull point which we started with. If we are not careful, we can include three collinear vertices on a hull edge, where in fact only the endpoints belong on the hull.

We resolve this by breaking ties in sorting by angle according to the distance from the initial hull point. By making sure the farthest of these collinear points is inserted last, we ensure that it remains on the final hull instead of its angular brethren:

```
bool smaller_angle(point *p1, point *p2)
{
        if (collinear(first_point,*p1,*p2)) {
                if (distance(first_point,*p1) <= distance(first_point,*p2))
                        return(-1);
                else
                        return(1);
        }

        if (ccw(first_point,*p1,*p2))
                return(-1);
        else
                return(1);
}
```

The remaining degenerate case concerns repeated points. What angle is defined between three occurrences of the same point? To eliminate this problem, we remove duplicate copies of points when we sort to identify the leftmost-lowest hull point:

```
sort_and_remove_duplicates(point in[], int *n)
{
        int i;                  /* counter */
        int oldn;               /* number of points before deletion */
        int hole;               /* index marked for potential deletion */
        bool leftlower();

        qsort(in, *n, sizeof(point), leftlower);

        oldn = *n;
```

```
        hole = 1;
        for (i=1; i<(oldn-1); i++) {
             if ((in[hole-1][X]==in[i][X]) && (in[hole-1][Y]==in[i][Y]))
                     (*n)--;
             else {
                     copy_point(in[i],in[hole]);
                     hole = hole + 1;
             }
        }
        copy_point(in[oldn-1],in[hole]);
}

bool leftlower(point *p1, point *p2)
{
        if ((*p1)[X] < (*p2)[X]) return (-1);
        if ((*p1)[X] > (*p2)[X]) return (1);

        if ((*p1)[Y] < (*p2)[Y]) return (-1);
        if ((*p1)[Y] > (*p2)[Y]) return (1);

        return(0);
}
```

There are a few final things to note about `convex_hull`. Observe the beautiful use of sentinels to simplify the code. We copy the origin point at the end of the insertion chain to avoid explicitly having to test for the wrap-around condition. We then implicitly delete this duplicated point by setting the return count appropriately.

Finally, note that we sort the points by angle without ever actually computing angles. The `ccw` predicate is enough to do the job.

14.4 Triangulation: Algorithms and Related Problems

Finding the perimeter of a polygon is easy; just compute the length of each edge using the Euclidean distance formula and add them together. Computing the area of irregular blobs is somewhat harder. The most straightforward approach is to divide the polygon into non-overlapping triangles and then sum the area of each triangle. The operation of partitioning a polygon into triangles is called *triangulation*.

Triangulating a convex polygon is easy, for we can just connect a given vertex v to all $n-1$ other vertices like a fan. This doesn't work for general polygons, however, because the edges might go outside the polygon. We must carve up a polygon P into triangles using non-intersecting chords which lie completely within P.

We can represent the triangulation either by listing the chords or, as we do here, with an explicit list of the vertex indices in each triangle.